FICHIERS ET MODULES

Lire, écrire et organiser

FICHIERS ET MODULES

📄 fichier ↔ 🐍 Python

LIRE UN FICHIER TEXTE

open, read et readlines

```python
# Lire tout le contenu d'un coup
f = open('notes.txt', 'r')
contenu = f.read()
print(contenu)
f.close()

# Lire ligne par ligne dans une liste
f = open('notes.txt', 'r')
lignes = f.readlines()
for ligne in lignes:
    print(ligne.strip())
f.close()
```

LIRE LIGNE PAR LIGNE

📄 fichier ↔ 🐍 Python

LE BLOC WITH

Le gardien qui ferme la porte

```python
# Le gardien qui ferme la porte derriere vous
with open('notes.txt', 'r') as f:
    contenu = f.read()
    print(contenu)
# Ici, f est automatiquement ferme

# Lire ligne par ligne (economise la memoire)
with open('notes.txt', 'r') as f:
    for ligne in f:
        print(ligne.strip())
```

WITH : GARDIEN AUTOMATIQUE

with : gardien automatique ✨ → propre

ÉCRIRE DANS UN FICHIER

Modes w et a

```python
# Ecrire (ecrase le fichier existant)
with open('resultat.txt', 'w') as f:
    f.write('Ligne 1\n')
    f.write('Ligne 2\n')

# Ajouter a la fin du fichier
with open('resultat.txt', 'a') as f:
    f.write('Ligne 3 ajoutee\n')

# Ecrire plusieurs lignes d'un coup
lignes = ['Alice: 18\n', 'Bob: 15\n', 'Clara: 20\n']
with open('notes_eleves.txt', 'w') as f:
    f.writelines(lignes)
```

ÉCRIRE DANS UN FICHIER

📄 fichier ↔ 🐍 Python

LE MODULE CSV

csv.reader et DictReader

```python
import csv

# csv.reader : chaque ligne est une liste
with open('eleves.csv', 'r') as f:
    lecteur = csv.reader(f)
    for ligne in lecteur:
        print(ligne)  # ['Alice', '18', 'Paris']

# csv.DictReader : chaque ligne est un dict
with open('eleves.csv', 'r') as f:
    lecteur = csv.DictReader(f)
    for eleve in lecteur:
        print(eleve['nom'], eleve['note'])
```

DONNÉES TABULAIRES CSV

	col_A	col_B	col_C
0	…	…	…
1	…	…	…
2	…	…	…

LE MODULE JSON

load, dump, loads, dumps

```python
import json

# Lire un fichier JSON
with open('config.json', 'r') as f:
    data = json.load(f)
    print(data['nom'])  # acces comme un dict

# Ecrire un fichier JSON
resultat = {'score': 95, 'eleve': 'Alice'}
with open('sortie.json', 'w') as f:
    json.dump(resultat, f, indent=2)

# Depuis/vers une chaine (le 's' = string)
texte = json.dumps(resultat)
```

STRUCTURE JSON

`"Structure JSON_1"` → valeur

`"Structure JSON_2"` → valeur

`"Structure JSON_3"` → valeur

CHEMINS ET DOSSIERS

os et pathlib

```python
import os
from pathlib import Path

# Verifier si un fichier existe
print(os.path.exists('notes.txt'))  # True/False

# Lister les fichiers d'un dossier
for nom in os.listdir('.'):
    print(nom)

# pathlib : la methode moderne
chemin = Path('data') / 'notes.txt'
print(chemin.exists())
print(chemin.suffix)  # .txt
```

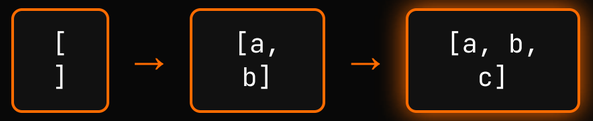
ARBORESCENCE FICHIERS

IMPORTER DES MODULES

import, from...import, pip

- import math → math.sqrt(16)
- from math import sqrt → sqrt(16)
- import numpy as np → alias pratique
- pip install pandas → installe depuis PyPI
- Vos propres fichiers .py sont des modules

IMPORT : BOÎTE À OUTILS



RÉCAPITULATIF

Fichiers et modules

- ✓ with open() : lecture et écriture sécurisées
- ✓ csv.reader / DictReader pour les tableaux
- ✓ json.load/dump (fichiers) vs loads/dumps (chaînes)
- ✓ pathlib pour les chemins modernes
- ✓ import et pip pour étendre Python